

# Evolve — The Late Evolutionary Layer

Preliminaries, a unified formalism, the design space, and related work

*A working document for organising ideas — no results*

Tom Offermann

ETH Zürich, Department of Computer Science

toffermann@ethz.ch

Version of September 21, 2026

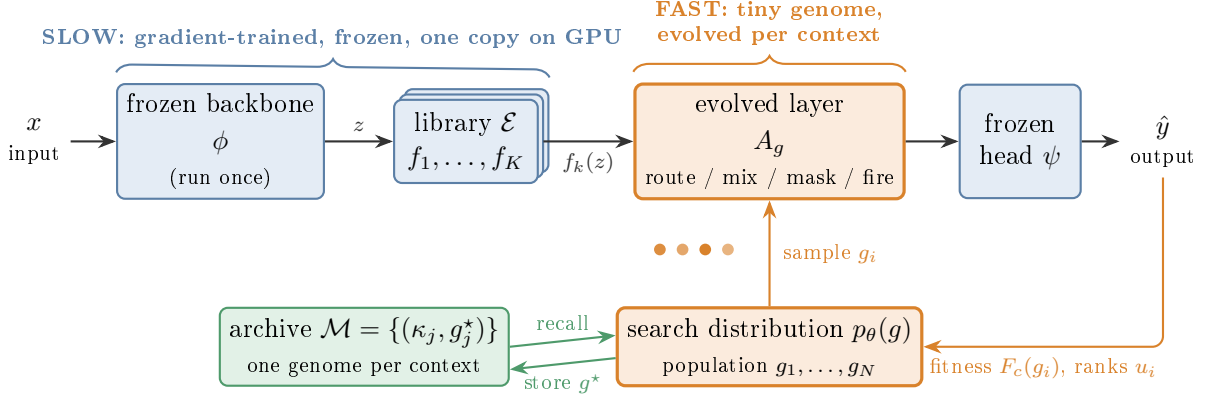
## The pitch in one paragraph

Today’s models are superb function approximators, but they are *frozen*: they do not adapt after deployment, and fine-tuning them overwrites what they knew. Evolutionary algorithms (EAs) are the opposite: poor at optimising  $10^9$  parameters, but excellent at fast, parallel, gradient-free search in *small*, possibly discrete, possibly non-differentiable spaces. The *late evolutionary layer* combines the two. All representational capacity lives in **slow** components (a pretrained backbone, a library of experts, a decoder) that are trained by gradients and then kept fixed. All adaptation lives in a **fast**, tiny *genome* ( $10^1$ – $10^5$  numbers or bits) placed *late* in the architecture, searched by an EA. Placing it late makes the population *separable*: one copy of the backbone, one shared forward pass, and  $N$  cheap evaluations of candidate genomes. Storing one genome per context yields a **memory** (an archive) that cannot forget, and recalling from it makes re-adaptation fast. This document fixes notation, formalises these claims, maps the design space, and places five concrete instantiations (F1–F5) in the literature.

## Contents

|          |                                                                             |          |
|----------|-----------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Motivation: slow components, fast genomes</b>                            | <b>3</b> |
| <b>2</b> | <b>Preliminaries and notation</b>                                           | <b>4</b> |
| 2.1      | Notation . . . . .                                                          | 4        |
| 2.2      | The optimisation problem . . . . .                                          | 4        |
| 2.3      | Evolution strategies as search-gradient methods . . . . .                   | 4        |
| 2.4      | Natural gradients: one framework for CMA-ES, ES and discrete EDAs . . . . . | 5        |
| 2.5      | Low-rank ES for weight matrices (EGGROLL) . . . . .                         | 5        |
| 2.6      | Discrete evolutionary algorithms . . . . .                                  | 6        |
| 2.7      | Quality-diversity and archives . . . . .                                    | 6        |
| <b>3</b> | <b>The late evolutionary layer</b>                                          | <b>6</b> |
| 3.1      | Definition . . . . .                                                        | 6        |
| 3.2      | Two kinds of separability . . . . .                                         | 6        |
| 3.3      | The cost model . . . . .                                                    | 7        |
| 3.4      | How good is an ES gradient in dimension $n$ ? . . . . .                     | 7        |
| 3.5      | The design space: where and how big . . . . .                               | 8        |
| <b>4</b> | <b>Continual adaptation with a genome archive</b>                           | <b>8</b> |
| 4.1      | Setting . . . . .                                                           | 8        |

|          |                                                               |           |
|----------|---------------------------------------------------------------|-----------|
| 4.2      | What forgetting means here . . . . .                          | 8         |
| 4.3      | Metrics . . . . .                                             | 10        |
| <b>5</b> | <b>Discrete genomes and runtime theory</b>                    | <b>10</b> |
| 5.1      | Re-optimisation after a small change . . . . .                | 10        |
| 5.2      | How far is reality from OneMax? Measuring epistasis . . . . . | 11        |
| 5.3      | A caveat that sharpens the question . . . . .                 | 11        |
| <b>6</b> | <b>The design space</b>                                       | <b>11</b> |
| <b>7</b> | <b>Five instantiations</b>                                    | <b>11</b> |
| <b>8</b> | <b>Related work</b>                                           | <b>13</b> |
| <b>9</b> | <b>Open questions</b>                                         | <b>14</b> |



**Figure 1: The late evolutionary layer.** Slow components (blue) are trained by gradient descent and then frozen; only one copy is held in memory and its outputs are shared by the whole population. The fast layer (orange) is parameterised by a tiny genome  $g$ , searched by an EA using any scalar fitness. The archive (green) stores the best genome per context and seeds future searches.

## 1 Motivation: slow components, fast genomes

A system that deserves the word *adaptive* should (i) learn continually without catastrophic forgetting, (ii) keep structured, reusable memory, (iii) adapt quickly to new situations on the basis of prior experience, and (iv) be able to change its own structure, not only its weights. Gradient-trained networks deliver none of these by default: they are trained once, deployed, and frozen, and every fine-tuning step moves all parameters at once.

The central design decision of this document is to *separate time scales*, in the spirit of complementary learning systems (a slow cortex and a fast hippocampus):

|           | slow system                   | fast system                                |
|-----------|-------------------------------|--------------------------------------------|
| what      | backbone, experts, decoder    | genome $g$ (routing, mixing, masks, rules) |
| size      | $10^8$ – $10^{10}$ parameters | $10^1$ – $10^5$ numbers or bits            |
| optimiser | gradient descent, pretraining | evolutionary algorithm                     |
| changes   | rarely (or never)             | per context, continuously                  |
| memory    | weights                       | archive of genomes                         |

Figure 1 shows the resulting architecture. The rest of this section explains, informally, *why* an EA belongs exactly in the orange box; Section 3 makes each point precise.

### Why the EA should be shallow and late (informal).

- Dimension.** Gradient estimates from a population of size  $N$  in dimension  $n$  carry a signal fraction of  $N/(N+n+1)$  (Proposition 3.5). At  $n = 10^9$  this is hopeless for any affordable  $N$ ; at  $n \leq 10^4$  it is excellent. Below  $n \approx 10^3$ , full-covariance CMA-ES learns the local metric of the problem and is often the best optimiser available at all.
- Adaptation is low-dimensional.** Fine-tuning a pretrained model succeeds in random subspaces of a few hundred to a few thousand dimensions (*intrinsic dimension*, Section 8). The *adaptation* problem is small even when the model is not.
- Separability.** If the genome only acts late, everything before it is identical across the population and is computed once (Proposition 3.3). The population then costs roughly one forward pass rather than  $N$ .

- (d) **No forgetting in the slow system.** The backbone never moves, so it cannot forget. All task-specific change is a stored genome that can be swapped or rolled back (Proposition 4.1).
- (e) **Hard decisions and non-differentiable objectives.** Routing, masking, thresholding, exact-match accuracy, worst-group accuracy, and episode returns are all non-differentiable. EAs need only a scalar per candidate.

## 2 Preliminaries and notation

### 2.1 Notation

| Symbol                                           | Meaning                                                                                                                                   |
|--------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| $x, y$                                           | input, target (label / answer / action)                                                                                                   |
| $c \in \mathcal{C}, \mathcal{D}_c$               | context (task, domain, damage condition) and its data distribution                                                                        |
| $\phi, z = \phi(x) \in \mathbb{R}^d$             | frozen backbone ( <b>slow</b> ) and its features                                                                                          |
| $\mathcal{E} = \{f_1, \dots, f_K\}$              | library of frozen experts / adapters / skills ( <b>slow</b> )                                                                             |
| $\psi$                                           | frozen head or decoder ( <b>slow</b> )                                                                                                    |
| $g \in \mathcal{G}, n = \dim \mathcal{G}$        | genome and its dimension ( <b>fast</b> ); $\mathcal{G} \subseteq \mathbb{R}^n$ or $\mathcal{G} = \Sigma^n$ for a finite alphabet $\Sigma$ |
| $A_g$                                            | evolved operator (routing, mixing, masking, firing) parameterised by $g$                                                                  |
| $\hat{y}_g(x) = \psi(A_g(\phi(x); \mathcal{E}))$ | model output under genome $g$                                                                                                             |
| $F_c(g)$                                         | fitness of $g$ in context $c$ (any scalar)                                                                                                |
| $\mathcal{R}_c, R =  \mathcal{R}_c $             | reference (few-shot) set used to estimate $F_c$                                                                                           |
| $p_\theta(g)$                                    | search distribution with parameters $\theta$                                                                                              |
| $N, g_1, \dots, g_N$                             | population size and population                                                                                                            |
| $u_i$                                            | rank-based utility of member $i$                                                                                                          |
| $\sigma, m, C$                                   | ES step size, mean, covariance                                                                                                            |
| $\mathcal{M} = \{(\kappa_j, g_j^*, \hat{F}_j)\}$ | archive: context key, best genome, its fitness ( <b>memory</b> )                                                                          |
| $L, \ell, \rho = (L - \ell)/L$                   | network depth, depth at which the genome acts, suffix fraction                                                                            |
| $B, C_{\text{layer}}, C_A$                       | batch size, cost of one layer, cost of the evolved operator                                                                               |

### 2.2 The optimisation problem

For a context  $c$  the fast system solves the black-box problem

$$g_c^* \in \arg \max_{g \in \mathcal{G}} F_c(g), \quad F_c(g) = \mathbb{E}_{(x,y) \sim \mathcal{D}_c} [r(\hat{y}_g(x), y)] - \beta \Omega(g), \quad (1)$$

where  $r$  is any per-example reward (accuracy, log-likelihood, exact match, return) and  $\Omega$  is a regulariser (e.g.  $\|g\|_1$  or a KL leash to the unmodified model). In practice  $F_c$  is estimated on a reference set  $\mathcal{R}_c$ :  $\hat{F}_c(g) = \frac{1}{R} \sum_{(x,y) \in \mathcal{R}_c} r(\hat{y}_g(x), y) - \beta \Omega(g)$ . Nothing requires  $r$ ,  $A_g$  or  $\psi$  to be differentiable.

### 2.3 Evolution strategies as search-gradient methods

Instead of optimising  $F$  directly, ES optimises the *expected* fitness under a search distribution,

$$J(\theta) = \mathbb{E}_{g \sim p_\theta} [F(g)], \quad \nabla_\theta J = \mathbb{E}_{g \sim p_\theta} [F(g) \nabla_\theta \log p_\theta(g)], \quad (2)$$

the *log-likelihood trick*, estimated by Monte Carlo over the population. For an isotropic Gaussian  $p_\theta = \mathcal{N}(m, \sigma^2 I)$  with  $g_i = m + \sigma \epsilon_i$  this becomes the familiar

$$\widehat{\nabla_m J} = \frac{1}{N\sigma} \sum_{i=1}^N u_i \epsilon_i, \quad m \leftarrow m + \alpha \widehat{\nabla_m J}. \quad (3)$$

Three standard variance reductions are used throughout:

- **Antithetic sampling:** evaluate  $m \pm \sigma \epsilon_i$  and use the difference  $\frac{1}{2}(F(m + \sigma \epsilon_i) - F(m - \sigma \epsilon_i))$ ; this cancels the constant and all even-order Taylor terms.
- **Rank-based utilities:** replace  $F(g_i)$  by  $u_i$ , a fixed function of its rank. This makes the update invariant to monotone transformations of  $F$  (accuracy vs. log-likelihood) and robust to outliers.
- **Common random numbers (CRN):** all members of a generation are evaluated on the *same* reference batch and the same environment seeds. Without CRN, differences between members are dominated by evaluation noise.

## 2.4 Natural gradients: one framework for CMA-ES, ES and discrete EDAs

The *information-geometric optimisation* (IGO) view takes the natural gradient  $\tilde{\nabla}_\theta J = \mathcal{I}(\theta)^{-1} \nabla_\theta J$  (with  $\mathcal{I}$  the Fisher information of  $p_\theta$ ), which makes the update independent of how  $\theta$  is parameterised:

$$\theta \leftarrow \theta + \eta \sum_{i=1}^N u_i \mathcal{I}(\theta)^{-1} \nabla_\theta \log p_\theta(g_i). \quad (4)$$

Different choices of  $p_\theta$  recover the algorithms used in this document:

| genome               | $p_\theta$                                      | algorithm                    | regime                                                         |
|----------------------|-------------------------------------------------|------------------------------|----------------------------------------------------------------|
| continuous, small    | $\mathcal{N}(m, \sigma^2 C)$                    | CMA-ES                       | $n \lesssim 10^3$ (full $C$ ), $\lesssim 10^5$ (diagonal $C$ ) |
| continuous, matrices | $\mathcal{N}(W, \sigma^2 I)$ , low-rank samples | EGGROLL-style ES             | up to $10^9$ , but see Proposition 3.5                         |
| discrete             | $\prod_j \text{Cat}(\text{softmax}(\theta_j))$  | PBIL / cGA / IGO-categorical | $10^3$ – $10^6$ bits or trits                                  |

**CMA-ES in brief.** Sample  $g_i = m + \sigma C^{1/2} \epsilon_i$ ; rank; with positive weights  $w_1 \geq \dots \geq w_\mu$  on the best  $\mu$  members,

$$m \leftarrow m + c_m \sum_{j=1}^\mu w_j (g_{j:N} - m), \quad (5)$$

$$C \leftarrow (1 - c_1 - c_\mu) C + c_1 p_c p_c^\top + c_\mu \sum_{j=1}^\mu w_j y_{j:N} y_{j:N}^\top, \quad y_{j:N} = (g_{j:N} - m)/\sigma, \quad (6)$$

with an evolution path  $p_c$  (rank-one update) and a separate path-length control of  $\sigma$ . CMA-ES learns a metric: correlated parameters receive correlated perturbations.

**Categorical IGO (PBIL-like).** For  $p_\theta(g_j = v) = \text{softmax}(\theta_j)_v$ , Equation (4) reduces to

$$\theta_{j,v} \leftarrow \theta_{j,v} + \eta \sum_{i=1}^N u_i \left( \mathbf{1}[g_{i,j} = v] - p_\theta(g_j = v) \right), \quad (7)$$

i.e. probabilities move towards the values used by good members.

## 2.5 Low-rank ES for weight matrices (EGGROLL)

For a weight matrix  $W \in \mathbb{R}^{a \times b}$ , a full Gaussian perturbation costs a separate full-rank matmul per member. Low-rank ES perturbs by  $E_i = \frac{\sigma}{\sqrt{r}} A_i B_i^\top$  with  $A_i \in \mathbb{R}^{a \times r}$ ,  $B_i \in \mathbb{R}^{b \times r}$ , so that

$$(W + E_i) u = W u + \frac{\sigma}{\sqrt{r}} A_i (B_i^\top u) :$$

the expensive  $W u$  is shared by all members, and each member adds only  $O(r(a+b))$ . The aggregate update  $\sum_i u_i A_i B_i^\top$  is still full rank. This is the *memory-separable* pattern of Definition 3.2 applied to the whole network; the late layer pushes it to its extreme.

## 2.6 Discrete evolutionary algorithms

- **(1+1) EA:** one parent; each position is mutated independently with probability  $1/n$  (to a uniformly random *other* symbol); the offspring replaces the parent if it is not worse.
- **$(\mu + \lambda)$  GA:**  $\mu$  parents create  $\lambda$  offspring by uniform crossover and mutation; the best  $\mu$  of parents  $\cup$  offspring survive.
- **Estimation-of-distribution algorithms** (cGA, PBIL, Equation (7)): keep a product distribution instead of a population; trivially parallel on GPUs.

## 2.7 Quality-diversity and archives

A *quality-diversity* (QD) algorithm returns many good solutions that behave differently. Given a behaviour descriptor  $b(g) \in \mathcal{B}$  and a partition of  $\mathcal{B}$  into cells, MAP-Elites keeps in each cell the best solution found so far. We will use the same data structure for *memory*: an archive whose cells are *contexts* rather than behaviours (Section 4).

## 3 The late evolutionary layer

### 3.1 Definition

**Definition 3.1** (Late evolutionary layer). A *late evolutionary layer* is a model of the form

$$\hat{y}_g(x) = \psi\left(A_g(h_{\leq \ell}(x); \mathcal{E})\right), \quad (8)$$

where  $h_{\leq \ell}$  denotes the first  $\ell$  of  $L$  layers of a frozen network (with  $\phi = h_{\leq \ell}$  as the backbone),  $\mathcal{E}$  and  $\psi$  are frozen, and only the genome  $g$  of the operator  $A_g$  is optimised, by an EA, with  $n = \dim \mathcal{G} \ll$  the number of frozen parameters. The layer is *late* if  $\rho = (L - \ell)/L$ , the fraction of computation downstream of the genome, is small.

The operator  $A_g$  covers all instantiations in this document:

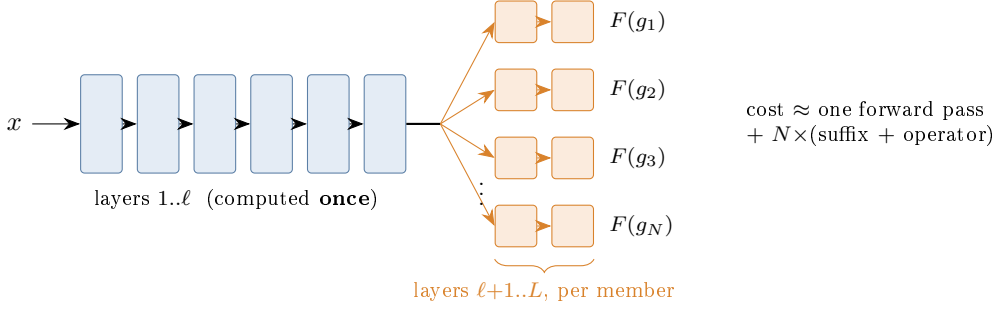
| operator        | genome                                            | example                                                                          |
|-----------------|---------------------------------------------------|----------------------------------------------------------------------------------|
| routing         | logit biases $b_\ell \in \mathbb{R}^K$            | $\tilde{s}_\ell(h) = W_\ell h + b_\ell$ , top- $k$ selection of experts (F1)     |
| mixing          | coefficients $\alpha \in \mathbb{R}^{G \times K}$ | $W' = W + \sum_k \alpha_{\gamma,k} B_k A_k$ over a LoRA library (F2)             |
| firing          | ternary weights $w_u \in \{-1, 0, 1\}^d$          | $h_u = \mathbf{1}[\langle w_u, z \rangle > \tau_u]$ , decoded by a codebook (F3) |
| selection/blend | descriptor target $d^*$ , gains $\kappa$          | policy = blend of repertoire skills near $d^*$ (F4)                              |
| plasticity      | rule parameters $\theta$                          | $\Delta W = \text{Rule}_\theta(o, z, M)$ , genome defines a learner (F5)         |

### 3.2 Two kinds of separability

**Definition 3.2** (Separability). A population evaluation of  $g_1, \dots, g_N$  on inputs  $x_1, \dots, x_B$  is

- **memory-separable** if all members share one copy of the frozen parameters, and each member adds at most  $O(n)$  memory;
- **compute-separable at depth  $\ell$**  if the activations  $h_{\leq \ell}(x_b)$  do not depend on  $g$ , so they can be computed once and reused by all  $N$  members.

Memory separability holds for every design in this document. Compute separability depends on the *task*:



**Figure 2: Compute separability.** If the genome acts only from depth  $\ell$  on and the input does not depend on it, the prefix is computed once and fanned out to  $N$  members. Only the orange suffix is paid  $N$  times.

| setting                                                     | memory-sep. | compute-sep.        |
|-------------------------------------------------------------|-------------|---------------------|
| classification, multiple-choice scoring (fixed input)       | ✓           | ✓                   |
| test-time adaptation on a fixed batch                       | ✓           | ✓                   |
| routing only at scored token positions of a fixed prompt    | ✓           | ✓                   |
| autoregressive generation (generated tokens depend on $g$ ) | ✓           | only for the prompt |
| closed-loop control (observations depend on actions)        | ✓           | ×                   |

### 3.3 The cost model

**Proposition 3.3** (Speed-up of a late layer). *Let each of  $L$  layers cost  $C_{\text{layer}}$  per input, let the operator cost  $C_A$  per input and member, and let the genome act from depth  $\ell$ . For a compute-separable evaluation of  $N$  members on  $B$  inputs,*

$$T_{\text{naive}} = NB(LC_{\text{layer}} + C_A), \quad (9)$$

$$T_{\text{sep}} = B\ell C_{\text{layer}} + NB((L - \ell)C_{\text{layer}} + C_A). \quad (10)$$

With  $\rho = (L - \ell)/L$  and  $\kappa = C_A/(LC_{\text{layer}})$ , the speed-up is

$$S = \frac{T_{\text{naive}}}{T_{\text{sep}}} = \frac{N(1 + \kappa)}{(1 - \rho) + N(\rho + \kappa)}, \quad \lim_{\rho, \kappa \rightarrow 0} S = N, \quad \lim_{N \rightarrow \infty} S = \frac{1 + \kappa}{\rho + \kappa}. \quad (11)$$

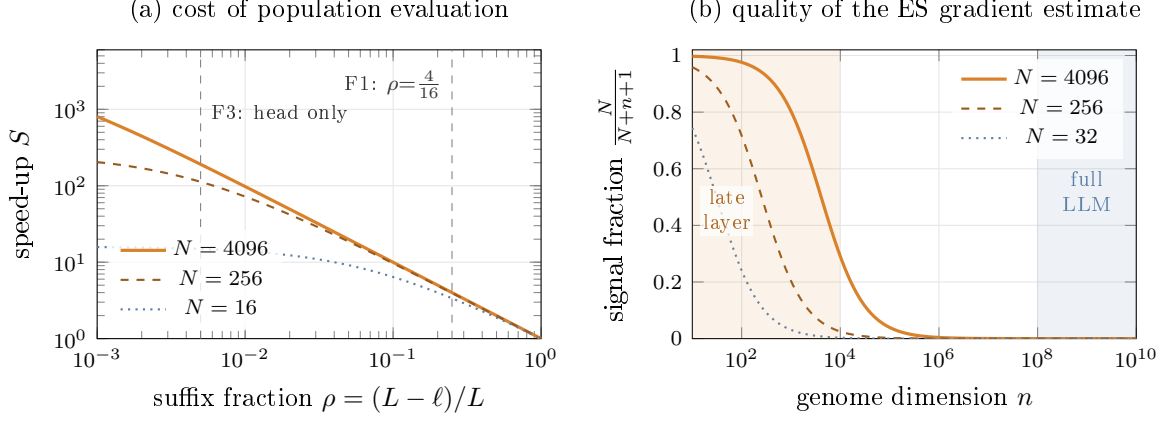
*Proof.* Naively, each of the  $N$  members runs all  $L$  layers and the operator on all  $B$  inputs. Under compute separability the first  $\ell$  layers are run once per input and the remaining  $L - \ell$  layers plus the operator once per member and input. Dividing, and substituting  $\ell = (1 - \rho)L$  and  $C_A = \kappa LC_{\text{layer}}$ , gives Equation (11); the limits are immediate.  $\square$

*Remark 3.4.* Two design rules follow. (i) The speed-up is capped by  $1/\rho$ : every layer after the genome costs a factor  $N$ . A frozen decoder  $\psi$  behind the evolved layer is only free if it is cheap or cacheable. (ii) The operator must be cheap compared to a layer ( $\kappa \ll \rho$ ); a mixing operator over  $K$  rank- $r$  adapters costs  $O(Kr(a + b))$  per token instead of  $O(ab)$ .

### 3.4 How good is an ES gradient in dimension $n$ ?

**Proposition 3.5** (Signal fraction of the ES estimator). *Let  $F(g) = \langle a, g \rangle$  be locally linear with  $a \in \mathbb{R}^n$ , and consider the antithetic estimator  $\hat{\nabla} = \frac{1}{2N\sigma} \sum_{i=1}^N (F(m + \sigma\epsilon_i) - F(m - \sigma\epsilon_i))\epsilon_i$  with  $\epsilon_i \stackrel{iid}{\sim} \mathcal{N}(0, I_n)$ . Then*

$$\mathbb{E}[\hat{\nabla}] = a, \quad \mathbb{E}[\|\hat{\nabla}\|^2] = \|a\|^2 \left(1 + \frac{n+1}{N}\right), \quad \frac{\|\mathbb{E}\hat{\nabla}\|^2}{\mathbb{E}\|\hat{\nabla}\|^2} = \frac{N}{N + n + 1}. \quad (12)$$



**Figure 3: Two reasons to go late and small.** (a) Proposition 3.3 with  $\kappa = 0$ : the speed-up approaches  $N$  as the genome moves to the end of the network. (b) Proposition 3.5: the fraction of the ES estimate that is signal is  $N/(N + n + 1)$ , excellent for  $n \leq 10^4$  and negligible at LLM scale.

In particular, for  $n \gg N$  the typical cosine between  $\hat{\nabla}$  and  $a$  is  $\approx \sqrt{N/n}$ .

*Proof.* For linear  $F$ ,  $\frac{1}{2\sigma}(F(m + \sigma\epsilon) - F(m - \sigma\epsilon)) = \langle a, \epsilon \rangle$ , so  $\hat{\nabla} = \frac{1}{N} \sum_i X_i$  with  $X_i = \epsilon_i \epsilon_i^\top a$  and  $\mathbb{E}X_i = a$ . Without loss of generality  $a = \|a\| e_1$ , so  $\mathbb{E}\|X_i\|^2 = \|a\|^2 \mathbb{E}[\epsilon_1^2 \|\epsilon\|^2] = \|a\|^2 (\mathbb{E}\epsilon_1^4 + (n - 1)) = \|a\|^2 (n + 2)$ . By independence,  $\mathbb{E}\|\hat{\nabla}\|^2 = \frac{1}{N} \mathbb{E}\|X_1\|^2 + \frac{N-1}{N} \|a\|^2 = \|a\|^2 (1 + \frac{n+1}{N})$ .  $\square$

*Remark 3.6.* This is the quantitative content of “EAs are slow in high dimensions”: the population must scale with  $n$  to keep the estimate informative. A late layer makes  $n$  small by design, so the same  $N$  that is useless for a full model is near-perfect for the genome.

### 3.5 The design space: where and how big

Figure 4 places existing work and our instantiations by genome size  $n$  and suffix fraction  $\rho$ . The lower-left region (small  $n$ , small  $\rho$ ) is where *both* Proposition 3.3 and Proposition 3.5 are favourable. Most existing gradient-free adaptation methods sit at  $\rho = 1$  (they perturb inputs or all layers); they are small in  $n$  but pay the full forward pass per member.

## 4 Continual adaptation with a genome archive

### 4.1 Setting

Contexts arrive as a stream  $c_1, c_2, \dots$  (possibly recurring or related). The slow system is fixed. The fast system maintains an archive

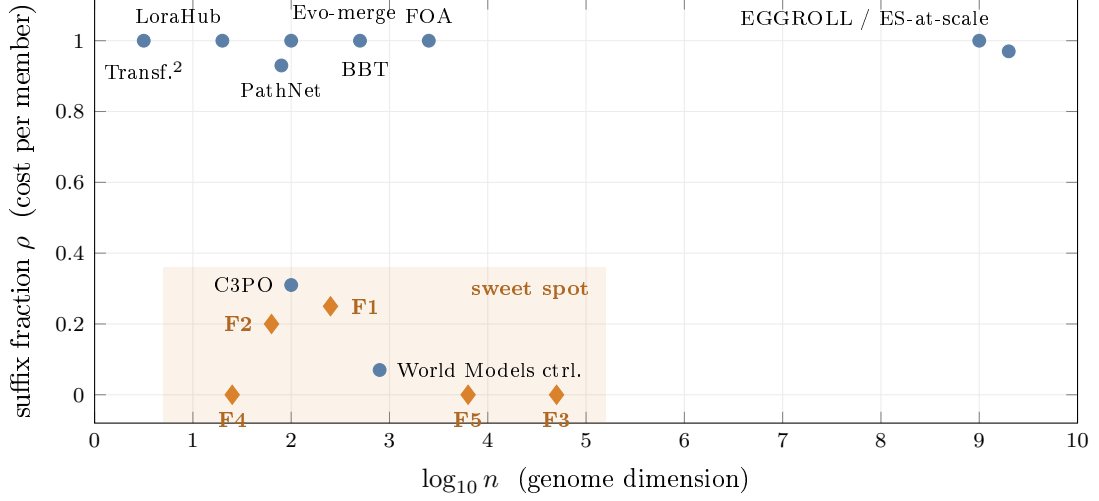
$$\mathcal{M} = \{(\kappa_j, g_j^*, \hat{F}_j)\}_j,$$

where  $\kappa_j$  is a *key* describing context  $j$ : for example the mean frozen feature  $\bar{z} = \frac{1}{R} \sum_{x \in \mathcal{R}_j} \phi(x)$ , or a behavioural descriptor. The archive is a MAP-Elites structure indexed by contexts rather than behaviours.

### 4.2 What forgetting means here

**Proposition 4.1** (No forgetting by construction). *If the slow system is frozen and, at test time, context  $c$  is served with the archived genome  $g_c^*$ , then the performance on  $c$  after any sequence of later contexts equals the performance at the time  $g_c^*$  was stored.*





**Figure 4: Design space.** Blue: existing gradient-free or evolutionary methods; orange: the instantiations F1–F5 of Section 7. Positions are indicative. Methods at  $\rho = 1$  pay a full forward pass per candidate; the shaded region is where separability and dimension are both favourable. Closed-loop settings (World Models, F4) are only memory-separable; their  $\rho$  refers to the controller’s share of computation.

---

**Algorithm 1:** Recall–adapt–store

---

**Input:** frozen  $\phi, \mathcal{E}, \psi$ ; archive  $\mathcal{M}$ ; new context  $c$  with reference set  $\mathcal{R}_c$ ; budget  $T$  generations of  $N$  members

---

```

1 Compute and cache the shared prefix  $h_{\leq \ell}(x)$  for all  $x \in \mathcal{R}_c$ ; // once
2  $\kappa_c \leftarrow$  key of  $c$  (e.g. mean prefix feature);
3 if  $\mathcal{M} \neq \emptyset$  then // recall
4    $\mathcal{Q} \leftarrow$  the  $M$  archive genomes with keys nearest to  $\kappa_c$ ;
5   evaluate all of  $\mathcal{Q}$  on  $\mathcal{R}_c$ ; // one batched population evaluation
6   initialise  $p_\theta$  at the best recalled genome, spread from the top recalled genomes;
7 else
8   initialise  $p_\theta$  at the identity genome (no modification)
9 for  $t = 1, \dots, T$  do // adapt
10  sample  $g_1, \dots, g_N \sim p_\theta$ ; evaluate  $\hat{F}_c(g_i)$  with CRN; update  $\theta$  by Equation (4);
11  $\mathcal{M} \leftarrow \mathcal{M} \cup \{(\kappa_c, g_c^*, \hat{F}_c(g_c^*))\}$ ; // store
12 return  $g_c^*$ 

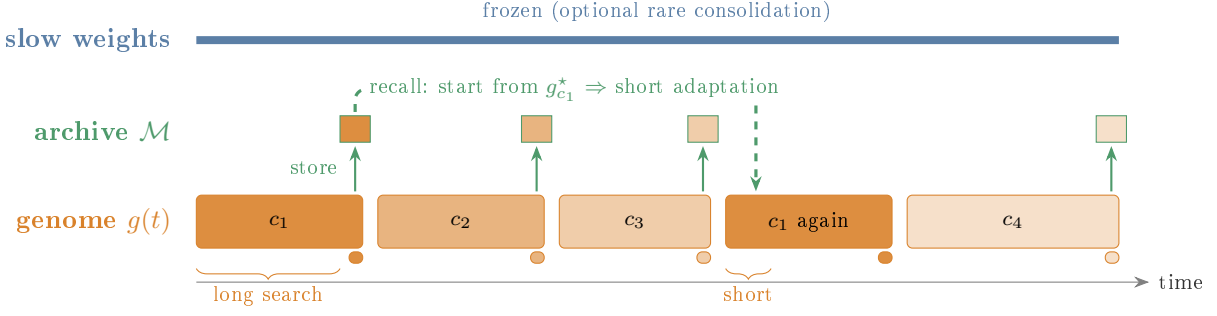
```

---

*Proof.*  $\hat{y}_{g_c^*}(x)$  depends only on the frozen  $\phi, \mathcal{E}, \psi$  and on  $g_c^*$ , none of which change after storing.  $\square$

The statement is trivial; its value is in isolating the *only two* remaining sources of forgetting:

1. **Key mis-recall.** If the context of a test input is unknown, it must be inferred from  $\kappa$ . Mis-recall rate is directly measurable.
2. **Interference in shared structure.** If new contexts extend a shared structure (e.g. new codewords in a shared decoder, F3), old contexts can be confused with new ones. This is decoding interference, not weight drift.



**Figure 5: Three time scales.** The slow weights never change. Each context is solved by a fast search of the genome; its result is stored in the archive. When a context recurs (or a similar one arrives), search starts from the recalled genome and is short.

### 4.3 Metrics

With  $A_{t,j}$  the accuracy on context  $j$  after learning context  $t$ , and  $T$  contexts:

$$\bar{A}_T = \frac{1}{T} \sum_{j=1}^T A_{T,j} \quad (\text{final average accuracy}) \quad (13)$$

$$\text{BWT} = \frac{1}{T-1} \sum_{j < T} (A_{T,j} - A_{j,j}) \quad (\text{backward transfer; } < 0 \text{ is forgetting}) \quad (14)$$

$$\text{AUC}_c(S) = \frac{1}{S} \sum_{s=1}^S F_c(g_{\text{best}}^{(s)}) \quad (\text{adaptation speed over the first } S \text{ evaluations}) \quad (15)$$

AUC on *revisited* vs. *new* contexts is the number that shows whether memory helps. A stream without recurring or related contexts cannot show it.

## 5 Discrete genomes and runtime theory

A discrete late layer (e.g. ternary threshold units, F3) makes the genome a string over a finite alphabet, which is exactly the object that EA runtime analysis studies. This section collects what transfers.

### 5.1 Re-optimisation after a small change

Let  $\mathcal{G} = \Sigma^n$  with  $|\Sigma| = s$ , and let  $g^*$  be a unique optimum. Call  $F$  *OneMax-like* if  $F(g) = -|\{j : g_j \neq g_j^*\}|$ , i.e. every wrong position costs the same and positions do not interact.

**Proposition 5.1** (Re-optimisation time from Hamming distance  $k$ ). *The  $(1+1)$  EA with per-position mutation probability  $1/n$  (to a uniformly random other symbol), started at Hamming distance  $k$  from  $g^*$  on a OneMax-like  $F$ , has expected optimisation time*

$$\mathbb{E}[T_k] \leq (s-1)enH_k = O(n \log k), \quad H_k = \sum_{i=1}^k \frac{1}{i}.$$

*Proof.* Fitness-level argument. At distance  $i$ , there are  $i$  wrong positions. An improvement occurs if exactly one of them is mutated, to the correct symbol, and nothing else changes; this has probability at least  $i \cdot \frac{1}{n} \cdot \frac{1}{s-1} \cdot (1 - \frac{1}{n})^{n-1} \geq \frac{i}{(s-1)en}$ . Elitism means the distance never increases, so the expected time to leave distance  $i$  is at most  $\frac{(s-1)en}{i}$ ; summing over  $i = 1..k$  gives the bound.  $\square$

*Remark 5.2.* A standard coupon-collector argument (each of the  $k$  wrong positions must be touched at least once) gives a matching  $\Omega(n \log k)$  lower bound for  $k$  not too small. The message for continual learning is that *re-adaptation time scales with the size of the change,  $\log k$ , not with the size of the model*. For general linear functions  $F(g) = \sum_j w_j \mathbf{1}[g_j = g_j^*]$ , tight  $O(n \log n)$  bounds from arbitrary initialisation are classical; a clean distance- $k$  bound for arbitrary positive weights is, as far as we know, not standard and is an open question within this project.

## 5.2 How far is reality from OneMax? Measuring epistasis

Real fitness functions (accuracy of threshold units, routing accuracy) are not linear. Define, at a solution  $g$  and for single-position changes  $\delta_i, \delta_j$ ,

$$\Delta_i^1 F(g) = F(g \oplus \delta_i) - F(g), \quad \Delta_{ij}^2 F(g) = F(g \oplus \delta_i \oplus \delta_j) - F(g \oplus \delta_i) - F(g \oplus \delta_j) + F(g), \quad (16)$$

and the *relative epistasis*  $\varepsilon(g) = \mathbb{E}_{i \neq j} |\Delta_{ij}^2 F(g)| / \mathbb{E}_i |\Delta_i^1 F(g)|$ .  $\varepsilon = 0$  is the additive (OneMax/linear) case where Proposition 5.1 applies. On cached features the fitness is deterministic, so  $\varepsilon$  can be measured exactly and cheaply.

## 5.3 A caveat that sharpens the question

For a single threshold unit  $h(z) = \mathbf{1}[\langle w, z \rangle > \tau]$  with targets  $t_b \in \{0, 1\}$ , the margin surrogate  $M(w) = \sum_b (2t_b - 1) \langle w, z_b \rangle = \langle w, v \rangle$ , with  $v = \sum_b (2t_b - 1) z_b$ , is linear and is maximised *in closed form* under a sparsity budget by keeping the largest  $|v_j|$  with  $w_j = \text{sign}(v_j)$ . No EA is needed for it. Any value an EA adds must therefore come from the non-linear part of the true objective (thresholding, class imbalance, feature interactions), which is exactly what  $\varepsilon$  quantifies.

## 6 The design space

Every instantiation is a point in a six-dimensional design space:

| axis                    | options                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------|
| <b>Where</b> ( $\rho$ ) | input side (prompts) / all layers / <b>last layers</b> / head only                      |
| <b>What</b> ( $A_g$ )   | routing / mixing / masking / firing / selection / learning rule                         |
| <b>Alphabet</b>         | continuous / binary / ternary / categorical (which expert, which path)                  |
| <b>Search</b>           | CMA-ES / low-rank ES / GA / EDA (PBIL, cGA) / QD (MAP-Elites)                           |
| <b>Fitness</b>          | likelihood / accuracy, exact match / worst-group / return / lifetime (prequential)      |
| <b>Memory</b>           | none / archive keyed by features / archive keyed by behaviour / evolved rule (implicit) |

Two further choices concern the *slow* side: whether the library is built for *diversity* (F2, F4) and whether the slow system is ever *consolidated* from frequently used genomes (distilling a genome into the backbone or spawning a new expert). The latter is the natural long-term outer loop and is out of scope here.

## 7 Five instantiations

Each instantiation has a self-contained proposal (`research/proposals/F-late-ea/F*.md`) with the full benchmark protocol. Here we give the idea and its central equation in the notation above.

### F1 — EvoRouter: evolved routing in a pretrained mixture-of-experts

A pretrained MoE language model is frozen. In the last  $L'$  MoE layers the router logits are shifted by a genome, and top- $k$  expert selection is kept *hard*:

$$\tilde{s}_\ell(h, x) = W_\ell h + b_\ell + U_\ell P \bar{e}(x), \quad \ell > L - L', \quad g = (b_\ell, U_\ell)_\ell, \quad n = KL'(1 + d'). \quad (17)$$

Top- $k$  has no useful gradient; the EA optimises the deployed discrete routing directly. Multiple-choice scoring is compute-separable with  $\rho = L'/L$ , and one genome per domain is archived.

## F2 — EvoCompose: composition of a diversified expert library

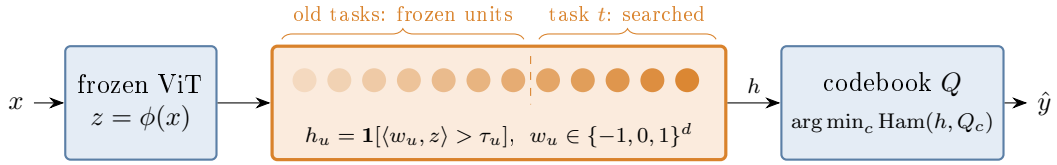
A library of  $K$  adapters is trained to be *expert* in-domain and *diverse* where the ensemble is uncertain:

$$\mathcal{L}_k = \mathbb{E}_{\mathcal{D}_k} [-\log p_k(y | x)] - \lambda \mathbb{E}_{x \sim \mathcal{D}_{\text{pool}}} \left[ (1 - a(x)) \frac{1}{K-1} \sum_{j \neq k} \omega_{kj}(x) \text{JS}(p_k(\cdot | x) \parallel \text{sg}[p_j(\cdot | x)]) \right], \quad (18)$$

with agreeableness  $a(x)$  (e.g. the best expert's confidence in the true label) and an expertness weight  $\omega_{kj}$ . Per task, a genome of mixing coefficients composes the library only in the last layers:

$$W'_\ell = W_\ell + \sum_{k=1}^K \alpha_{\gamma(\ell),k} B_{\ell,k} A_{\ell,k}, \quad g = (\alpha_{\gamma,k}) \in \mathbb{R}^{G \times K}. \quad (19)$$

## F3 — The Sandwich: frozen encoder, evolved discrete firing layer, frozen decoder



Units decompose: for fixed codewords each unit  $u$  has target bits  $t_{b,u} = Q_{y_b,u}$  and an independent fitness

$$F_u(w_u) = \frac{1}{B} \sum_b \mathbf{1}[\langle w_u, z_b \rangle > \tau_u] = t_{b,u}, \quad (20)$$

so  $m$  searches run in parallel on cached features at  $10^4$ – $10^6$  evaluations per second, with  $\tau_u$  set optimally by one sort. New tasks add units; old units never change. This is the natural test-bed for Proposition 5.1 and the epistasis measure  $\varepsilon$ .

## F4 — Repertoire adaptation inside a learned model

Offline, MAP-Elites builds a repertoire  $\{(\pi_j, d_j, f_j)\}$  of diverse skills. After a change (e.g. robot damage) a small genome blends skills near a target descriptor and adds feedback,

$$\pi_g(s) = \sum_{j \in \mathcal{N}_q(d^*)} w_j(g) \pi_j(s) + K_\kappa \phi_{\text{fb}}(s), \quad g = (d^*, \log T, \kappa), \quad (21)$$

and CMA-ES evaluates the population inside a learned residual model  $\hat{s}_{t+1} = \text{Sim}(s_t, a_t) + \delta_\omega(s_t, a_t)$ , with a pessimistic fitness  $\hat{F}(g) = \text{mean}_e R_e(g) - \beta \text{std}_e R_e(g)$  over an ensemble. The population lives in the model; the real system only sees the champion.

## F5 — Evolved plasticity

The EA moves one level up and searches the parameters  $\theta$  of a local learning rule for the late layer,

$$\Delta W_t = \eta \odot \left[ M_t (A \odot o_t z_t^\top + B \odot o_t \mathbf{1}^\top + C \odot \mathbf{1} z_t^\top + D) + E \odot e_t z_t^\top \right] - \lambda \odot W_t, \quad (22)$$

optimising the prequential lifetime objective  $\mathbb{E}_c [\frac{1}{T} \sum_t \text{perf}(\hat{y}_t, y_t)]$  over a distribution of drifting contexts. The error term  $E \odot e_t z_t^\top$  makes online SGD a member of the family, so an evolved rule must beat it to matter.

## At a glance

|    | genome                          | $n$                   | separability                   | EA-specific advantage                     |
|----|---------------------------------|-----------------------|--------------------------------|-------------------------------------------|
| F1 | router biases (+ conditioning)  | $10^2$ – $10^4$       | compute ( $\rho = L'/L$ )      | hard top- $k$ routing                     |
| F2 | mixing coefficients             | $10^1$ – $10^3$       | compute (late layers)          | multi-modal mixing; population = ensemble |
| F3 | ternary unit weights            | $10^4$ – $10^5$ trits | compute ( $\rho \approx 0$ )   | discrete; theory; massive throughput      |
| F4 | descriptor target, blend, gains | $10^1$ – $10^2$       | memory only; parallel in model | noisy, multi-modal model fitness          |
| F5 | rule parameters                 | $10^1$ – $10^4$       | compute over contexts          | long, non-differentiable lifetimes        |

## 8 Related work

**Evolution strategies at scale.** ES as a scalable alternative to RL [45] and deep neuroevolution with GAs [48] showed that population methods can train deep networks given enough parallel compute. EGGROLL [10] makes ES on billion-parameter models practical through low-rank perturbations, and ES has been shown competitive with RL for LLM fine-tuning [43], with the caveat that its dense, high-norm updates can cause forgetting [11]. These are the “deep” end of Figure 4; this document argues for the “shallow and late” end. CMA-ES [16], natural evolution strategies [54] and IGO [38] provide the algorithmic backbone.

**Frozen features with a small evolved controller.** World Models [15] trains a VAE and a recurrent model by gradients and then evolves an 867-parameter controller with CMA-ES — the clearest precedent for the slow/fast split. PathNet [12] uses a GA to select paths through layers of modules and freezes the paths of earlier tasks, an early instance of evolved, memory-backed continual learning.

**Low-dimensional, gradient-free adaptation.** Fine-tuning has a small intrinsic dimension [24, 2]. Black-box tuning optimises a low-dimensional prompt projection with CMA-ES [50]. FOA adapts a frozen vision transformer at test time with CMA-ES on input prompts, using forward passes only [37].

**Composing and merging experts.** Task arithmetic [18] and learned merging coefficients [57] show that fine-tuned deltas compose. LoraHub [17] searches LoRA mixing weights gradient-free from a few examples; Transformer<sup>2</sup> [51] mixes singular-value expert vectors at test time, including by CEM. Evolutionary model merging [3], CycleQD [22] and M2N2 [1] evolve merge recipes and maintain diversity through QD or resource competition. Libraries of domain experts can be built by branching and training separately [25, 49] and routed without search [39]. Methods that train models to disagree out of distribution [23, 40] and population-level diversity objectives [41] motivate the diversity-regularised library of F2.

**Mixture-of-experts routing.** C3PO [26] re-mixes expert weights in the last layers of pre-trained MoE models at test time using neighbour-based surrogates and recovers a large part of an oracle routing gap. Evolutionary expert pruning [28] and auxiliary-loss-free bias-based load balancing [53] show, respectively, that EAs apply to MoE structure and that per-expert biases are a meaningful low-dimensional control. OLMoE [35] is a fully open testbed.

**Discrete computation and masks.** Binary masks over fixed random weights (supermasks) reach good accuracy [58, 44], and one mask per task yields continual learning without forgetting

[56, 30]. Weight-agnostic networks [13], Tsetlin machines [14], logic-gate networks [42] and ternary LLMs [29] show that discrete computation is viable. Frozen pretrained features with simple heads are strong continual learners [59, 31, 52], and last-layer retraining is sufficient for robustness to spurious correlations [20].

**Fast adaptation from a repertoire.** Intelligent Trial and Error [8] builds a MAP-Elites [34] repertoire offline and adapts to damage online with Bayesian optimisation over it; follow-ups add reset-free operation [7] and selection among repertoires [19]. Black-DROPS [6] runs CMA-ES inside learned dynamics models. QDax [27, 5] makes QD massively parallel on accelerators.

**Evolved plasticity and meta-learning.** Evolved Hebbian rules let randomly initialised networks self-organise within an episode and cope with unseen damage [36]; differentiable plasticity and neuromodulation [32, 33] and fast weights [4] are the gradient-based counterparts. Context-vector meta-learning [60] and ES-based MAML [47] adapt only a small set of parameters.

**Runtime theory.** The (1+1) EA optimises linear functions in  $\Theta(n \log n)$  [9, 55]; dynamic and re-optimisation settings have been analysed for OneMax variants [21] and linear functions under dynamic constraints [46]. Section 5 asks how far these results carry to a discrete late layer.

**Gap.** Existing work either (i) evolves a small late component *once* (merging recipes, controllers), (ii) adapts at test time *without memory* (FOA, C3PO, Transformer<sup>2</sup>), or (iii) learns continually with *gradient*-found masks or prompts (SupSup, L2P). The combination studied here — a frozen or diversified library, a late and separable evolved layer exploiting massive parallelism, and an archive of genomes as memory, evaluated on adaptation speed and forgetting — is, to our knowledge, open.

## 9 Open questions

1. **Where is the cut?** How small can  $\rho$  be before the frozen components cap what adaptation can reach? Is there a principled way to choose  $\ell$  (e.g. from layer-wise task specificity)?
2. **Diversity of the slow system.** Does a library trained for diversity make the fast search easier? Can this be quantified, e.g. via the effective rank of per-example correctness vectors across experts?
3. **Keys and recall.** Which context keys make recall reliable? When does soft blending of archived genomes beat hard recall?
4. **Theory beyond OneMax.** Distance- $k$  re-optimisation bounds for general linear functions; bounds in terms of measured epistasis  $\varepsilon$ .
5. **Consolidation.** When and how should frequently recalled genomes be distilled into the slow system, and does that re-introduce forgetting?
6. **Search at deployment vs. learned learning.** When does an evolved rule (F5) replace search plus archive (F1–F4), and can they be combined hierarchically?

## References

- [1] J. Abrantes, R. Lange, Y. Tang. Competition and attraction improve model fusion (M2N2). GECCO 2025. arXiv:2508.16204.
- [2] A. Aghajanyan, L. Zettlemoyer, S. Gupta. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. ACL 2021. arXiv:2012.13255.

- [3] T. Akiba, M. Shing, Y. Tang, Q. Sun, D. Ha. Evolutionary optimization of model merging recipes. *Nature Machine Intelligence* 2025. arXiv:2403.13187.
- [4] J. Ba, G. Hinton, V. Mnih, J. Leibo, C. Ionescu. Using fast weights to attend to the recent past. *NeurIPS* 2016. arXiv:1610.06258.
- [5] F. Chalumeau et al. QDax: a library for quality-diversity and population-based algorithms with hardware acceleration. *JMLR* 2024. arXiv:2308.03665.
- [6] K. Chatzilygeroudis, R. Rama, R. Kaushik, D. Goepp, V. Vassiliades, J.-B. Mouret. Black-box data-efficient policy search for robotics. *IROS* 2017. arXiv:1703.07261.
- [7] K. Chatzilygeroudis, V. Vassiliades, J.-B. Mouret. Reset-free trial-and-error learning for robot damage recovery. *Robotics and Autonomous Systems* 2018. arXiv:1610.04213.
- [8] A. Cully, J. Clune, D. Tarapore, J.-B. Mouret. Robots that can adapt like animals. *Nature* 521, 2015. arXiv:1407.3501.
- [9] S. Droste, T. Jansen, I. Wegener. On the analysis of the (1+1) evolutionary algorithm. *Theoretical Computer Science* 276, 2002.
- [10] Evolution strategies at the hyperscale (EGGROLL). arXiv:2511.16652, 2025.
- [11] Evolutionary strategies lead to catastrophic forgetting in LLMs. arXiv:2601.20861, 2026.
- [12] C. Fernando et al. PathNet: evolution channels gradient descent in super neural networks. arXiv:1701.08734, 2017.
- [13] A. Gaier, D. Ha. Weight agnostic neural networks. *NeurIPS* 2019. arXiv:1906.04358.
- [14] O.-C. Granmo. The Tsetlin machine. arXiv:1804.01508, 2018.
- [15] D. Ha, J. Schmidhuber. World models / Recurrent world models facilitate policy evolution. *NeurIPS* 2018. arXiv:1803.10122.
- [16] N. Hansen. The CMA evolution strategy: a tutorial. arXiv:1604.00772, 2016.
- [17] C. Huang, Q. Liu, B. Y. Lin, T. Pang, C. Du, M. Lin. LoraHub: efficient cross-task generalization via dynamic LoRA composition. *COLM* 2024. arXiv:2307.13269.
- [18] G. Ilharco et al. Editing models with task arithmetic. *ICLR* 2023. arXiv:2212.04089.
- [19] R. Kaushik, P. Desreumaux, J.-B. Mouret. Adaptive prior selection for repertoire-based online adaptation in robotics. *Frontiers in Robotics and AI* 2020. arXiv:1907.07029.
- [20] P. Kirichenko, P. Izmailov, A. G. Wilson. Last layer re-training is sufficient for robustness to spurious correlations. *ICLR* 2023. arXiv:2204.02937.
- [21] T. Kötzing, A. Lissovoi, C. Witt. (1+1) EA on generalized dynamic OneMax. *FOGA* 2015.
- [22] S. Kuroki, T. Nakamura, T. Akiba, Y. Tang. Agent skill acquisition for LLMs via CycleQD. *ICLR* 2025. arXiv:2410.14735.
- [23] Y. Lee, H. Yao, C. Finn. Diversify and disambiguate: learning from underspecified data. *ICLR* 2023. arXiv:2202.03418.
- [24] C. Li, H. Farkhoor, R. Liu, J. Yosinski. Measuring the intrinsic dimension of objective landscapes. *ICLR* 2018. arXiv:1804.08838.
- [25] M. Li et al. Branch-train-merge: embarrassingly parallel training of expert language models. arXiv:2208.03306, 2022.
- [26] Z. Li, Z. Li, T. Zhou. C3PO: critical-layer, core-expert, collaborative pathway optimization for test-time expert re-mixing. arXiv:2504.07964, 2025.
- [27] B. Lim, M. Allard, L. Grillotti, A. Cully. Accelerated quality-diversity through massive parallelism. *TMLR* 2023. arXiv:2202.01258.

- [28] E. Liu et al. Efficient expert pruning for sparse mixture-of-experts language models (EEP). arXiv:2407.00945, 2024.
- [29] S. Ma et al. The era of 1-bit LLMs: all large language models are in 1.58 bits. arXiv:2402.17764, 2024.
- [30] M. Masana, T. Tuytelaars, J. van de Weijer. Ternary feature masks: zero-forgetting for task-incremental learning. CVPR Workshops 2021. arXiv:2001.08714.
- [31] M. D. McDonnell, D. Gong, A. Parvaneh, E. Abbasnejad, A. van den Hengel. RanPAC: random projections and pre-trained models for continual learning. NeurIPS 2023. arXiv:2307.02251.
- [32] T. Miconi, J. Clune, K. O. Stanley. Differentiable plasticity. ICML 2018. arXiv:1804.02464.
- [33] T. Miconi, A. Rawal, J. Clune, K. O. Stanley. Backpropamine: training self-modifying neural networks with differentiable neuromodulated plasticity. ICLR 2019. arXiv:2002.10585.
- [34] J.-B. Mouret, J. Clune. Illuminating search spaces by mapping elites. arXiv:1504.04909, 2015.
- [35] N. Muennighoff et al. OLMoE: open mixture-of-experts language models. arXiv:2409.02060, 2024.
- [36] E. Najarro, S. Risi. Meta-learning through Hebbian plasticity in random networks. NeurIPS 2020. arXiv:2007.02686.
- [37] S. Niu, C. Miao, G. Chen, P. Wu, P. Zhao. Test-time model adaptation with only forward passes. ICML 2024. arXiv:2404.01650.
- [38] Y. Ollivier, L. Arnold, A. Auger, N. Hansen. Information-geometric optimization algorithms: a unifying picture via invariance principles. JMLR 2017. arXiv:1106.3708.
- [39] O. Ostapenko et al. Towards modular LLMs by building and reusing a library of LoRAs. ICML 2024. arXiv:2405.11157.
- [40] M. Pagliardini, M. Jaggi, F. Fleuret, S. P. Karimireddy. Agree to disagree: diversity through disagreement for better transferability. ICLR 2023. arXiv:2202.04414.
- [41] J. Parker-Holder, A. Pacchiano, K. Choromanski, S. Roberts. Effective diversity in population-based reinforcement learning. NeurIPS 2020. arXiv:2002.00632.
- [42] F. Petersen, C. Borgelt, H. Kuehne, O. Deussen. Deep differentiable logic gate networks. NeurIPS 2022. arXiv:2210.08277.
- [43] X. Qiu et al. Evolution strategies at scale: LLM fine-tuning beyond reinforcement learning. arXiv:2509.24372, 2025.
- [44] V. Ramanujan, M. Wortsman, A. Kembhavi, A. Farhadi, M. Rastegari. What’s hidden in a randomly weighted neural network? CVPR 2020. arXiv:1911.13299.
- [45] T. Salimans, J. Ho, X. Chen, S. Sidor, I. Sutskever. Evolution strategies as a scalable alternative to reinforcement learning. arXiv:1703.03864, 2017.
- [46] F. Shi, M. Schirneck, T. Friedrich, T. Kötzing, F. Neumann. Reoptimization time analysis of evolutionary algorithms on linear functions under dynamic uniform constraints. Algorithmica 81, 2019.
- [47] X. Song et al. ES-MAML: simple Hessian-free meta learning. ICLR 2020. arXiv:1910.01215.
- [48] F. P. Such et al. Deep neuroevolution: genetic algorithms are a competitive alternative for training deep neural networks for reinforcement learning. arXiv:1712.06567, 2017.
- [49] S. Sukhbaatar et al. Branch-train-MiX: mixing expert LLMs into a mixture-of-experts LLM. arXiv:2403.07816, 2024.
- [50] T. Sun, Y. Shao, H. Qian, X. Huang, X. Qiu. Black-box tuning for language-model-as-a-service. ICML 2022. arXiv:2201.03514.
- [51] Q. Sun, E. Cetin, Y. Tang. Transformer<sup>2</sup>: self-adaptive LLMs. ICLR 2025. arXiv:2501.06252.
- [52] Z. Wang et al. Learning to prompt for continual learning. CVPR 2022. arXiv:2112.08654.



- [53] L. Wang, H. Gao, C. Zhao, X. Sun, D. Dai. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.
- [54] D. Wierstra, T. Schaul, T. Glasmachers, Y. Sun, J. Peters, J. Schmidhuber. Natural evolution strategies. JMLR 2014. arXiv:1106.4487.
- [55] C. Witt. Tight bounds on the optimization time of a randomized search heuristic on linear functions. Combinatorics, Probability and Computing 22, 2013.
- [56] M. Wortsman et al. Supermasks in superposition. NeurIPS 2020. arXiv:2006.14769.
- [57] E. Yang et al. AdaMerging: adaptive model merging for multi-task learning. ICLR 2024. arXiv:2310.02575.
- [58] H. Zhou, J. Lan, R. Liu, J. Yosinski. Deconstructing lottery tickets: zeros, signs, and the supermask. NeurIPS 2019. arXiv:1905.01067.
- [59] D.-W. Zhou, Z.-W. Cai, H.-J. Ye, D.-C. Zhan, Z. Liu. Revisiting class-incremental learning with pre-trained models: generalizability and adaptivity are all you need. IJCV 2024. arXiv:2303.07338.
- [60] L. Zintgraf, K. Shiarlis, V. Kurin, K. Hofmann, S. Whiteson. Fast context adaptation via meta-learning (CAVIA). ICML 2019. arXiv:1810.03642.